Contents

1 Introduction **2**

2 Problem Statement **3**

3 Methodology **3**
3.1 Dataset Preparation 4
3.2 Simulation Environment 4
3.3 Algorithm Implementation 4
3.4 Experimental Design 5
3.5 Performance Metrics 5

4 Output and Result Analysis **5**
4.1 Effect of Workload Size 6
4.2 Effect of Burst Time Variance 8
4.3 Effect of Time Quantum 9
4.4 Effect of Arrival Patterns 10
4.5 Context Switching Comparison 11
4.6 Overall Observations 12

5 Conclusion **12**

Abstract

The scheduling of CPU is one of the requirements of a modern operating system that ensures equitable execution of processes, minimizes the waiting time, and maximizing the utilization of the system. Classical scheduling algorithms, including Round Robin (RR), are fair and can be highly inefficient in cases where the workload of processes is highly varied. Adaptive rescheduling methods like Adaptive Dynamic Round Robin (ADRR) are trying to overcome these constraints by dynamically changing the time quantum with workload properties. Recently, the use of machine learning methods has been pursued. The paper gives an element of comparative analysis of three scheduling methods, Round Robin (RR), and Adaptive Dynamic Round Robin (ADRR), and an Artificial Intelligence-enhanced Adaptive Dynamic Round Robin (AI-ADRR) algorithm. The algorithms are tested under various working situations comprising different strengths of workload, variance in burst time, time quantum values, and variances in arrival patterns. Synthetic datasets in the form of scheduling environments that resembles potential situations are used to conduct experimental simulations. Analysis of performance measures such as the average waiting time, turnaround-time, response-time, CPU-utilization and context switches are presented. The findings indicate that adaptive scheduling methods enhance the performance of the scheduling as compared to conventional RR, especially in the case of heterogeneous workloads. Moreover, AI-ADRR with the integration of machine learning offers further improvements under certain conditions of a workload through its ability to better estimate burst times and improve time quantum adjustment.

1 Introduction

The modern operating systems have the task of controlling various processes that are competing over scarce computational resources. CPU scheduling is one of the most imperative tasks of an operating system, which changes the sequence according to which processes will be run. Sound scheduling is one that leads to an optimal use of resources, as well as being fair and limiting the delays incurred by processes. The Round Robin (RR) algorithm is a well-known method of scheduling in time-sharing systems as it is fair and simple. In the case of RR scheduling, a process is allocated a constant time quantum in which they can be run. In case the process is not completed within this time then it is preempted and sent back to the ready queue. Even though this strategy helps to avoid starvation and provides fairness, the efficiency of this strategy highly relies on the time quantum used. When the quantum is made too small, then excessive context switching is done and when made too large it functions in much the same way as First-Come-First-Served scheduling. In order to overcome that, various adaptive scheduling techniques are offered. The Adaptive Dynamic Round Robin (ADRR) algorithm is one of such techniques, and it modifies the time quantum in response to attributes of processes currently in the ready queue. ADRR tries to find a compromise between fairness and efficiency of performance by dynamically changing quantum. Current developments in the field of artificial intelligence have presented new options in enhancing the selection of CPU scheduling. Historical process properties can be analyzed by machine learning models and used to predict burst times, which will allow smarter scheduling policies. The AI-ADRR method enables the scheduler to more effectively adapt to heterogeneous workloads by incorporating the machine learning propositions in the dynamical quantum selection process. This report explores the behavior of RR, ADRR and AI-ADRR

scheduling algorithms in a different work load setting. The purpose of this is to know the reaction of adaptive and intelligent scheduling schemes to alterations in the nature of systems and to assess whether AI-based methods offer quantifiable performances of scheduling performance.

2 Problem Statement

The traditional algorithms of CPU scheduling usually presuppose the comparatively steady workload of processes. Nonetheless, there are often dynamic processes in real-world systems that have classes of base phenomenon workloads with different process arrival patterns, burst intervals, and work loads. In this case, hard scheduling parameters can contribute to the ineffective use of resources and the problem of the lengthening of the waiting period.

Although Round Robin algorithm is easy and even-handed, it has a number of shortcomings when used on the heterogeneous workloads. Its requirement to rely on a fixed time quantum may result in excessive context switching in cases when quantum is small, or poor responsiveness in cases when quantum is large. These problems are further increased when process burst times are greatly different.

Adaptive techniques like ADRR aim at alleviating these challenges by dynamically adjusting the time quantum on the basis of the readiness queue properties. Although it enhances flexibility, ADRR continues to use methods of heuristics that are not always able to reveal the reality of process workloads.

It can be assumed that with the introduction of machine learning methods, newly created predictive models can be implemented to augment scheduling decisions. An AI-enhanced scheduler may potentially pick a more efficient allocation of CPU resources by estimating burst times in the future based on the historical data. Therefore, the central objective of this study is to evaluate the following research questions:

- How does workload size affect the performance of RR, ADRR, and AI-ADRR scheduling algorithms?
- What is the impact of burst time variance on scheduling efficiency?
- How does the choice of time quantum influence system performance?
- How do different process arrival patterns affect the comparative performance of scheduling algorithms?

By systematically analyzing these factors, the study aims to determine whether AI-enhanced scheduling provides measurable benefits over traditional and adaptive scheduling techniques.

3 Methodology

The testing of scheduling algorithm was conducted on a simulation-based system coded in Python. A simulation environment uses these mechanisms to simulate the behaviour of a CPU scheduler by holding a list of processes waiting to be run, and run them based on the specifications that each scheduling algorithm has. The main aim of the methodology was to have fair comparison by executing the same workloads on all the algorithms but differing only on the logic of scheduling.

3.1 Dataset Preparation

The initial step entailed preparation of datasets of varying workload conditions. There were four attributes in every process in the dataset, namely process identifier (PID), arrival time, burst time, and priority value. These parameters were used to make the simulation close to real operating system processes behaviour. In the experiments with high workload, datasets of 500 processes were created. The times of arrival were also spread at various ranges as per the experiment scenario. The uniform arrival dataset had processes evenly distributed between time 0 and 2495 forming a continuous process inflow. In burst arrival events, each process reached time 0 which simulates a high demand on the CPU. Processes within the limited time window were introduced in random arrival datasets to simulate unforeseen workloads in systems.

3.2 Simulation Environment

The algorithms in the implementation of the scheduling were conducted in a Python simulation environment that had a dynamic ready queue of processes. The progression was represented as a system clock variable. of time during execution. At every time step, the scheduler took one process in the ready queue based on the scheduling policy and assigned CPU time to the process. There are various performance measures that the simulation monitored during the execution. Waiting time was computed as the length of time that a process had to stay in the ready queue before it was given access to a CPU. The turnaround time was calculated as the overall time of process arrival and the process completion. Response time was the time wasted before a process entered the system and had the first allocation of the CPU.

3.3 Algorithm Implementation

There were three scheduling algorithms that were applied in the simulation structure. The initial one was the conventional Round Robin scheduler. A quantum time is also specified with each processor in this algorithm. In case a process fails to execute its quantum length, it is preempted and put at the tail of the ready queue. The latter algorithm was Adaptive Dynamic Round Robin (ADRR). Hard wired quan- tum values are manipulated dynamically by ADRR with the time quantum dynamically treading the burst-time properties of the processes currently in play in the ready queue. In this adaptive strategy, longer processes are able to run without being interrupted too often thus minimizing context switching overheads. The third algorithm was the AI-ADRR that builds on ADRR concept with introduction of predictive decision making. The algorithm examines work load trends and determines a schedule behaviour according to the requirements of process execution. This strategy tries to maximize further the CPU usage and any delays in scheduling.

3.4 Experimental Design

To test the algorithms when the workload conditions vary, four experimental situations were prepared. The initial experiment examined the size of workload through the implementation of great amounts of processes at the same time. The second experiment was devoted to the impact of burst time variance creating datasets of low variability, moderate variability, and high variability. The third experiment examined the effect of values of time quantum on the performance of Round Robin scheduling. In the last experiment the impact of various process arrival patterns on the scheduling efficiency was tested.

3.5 Performance Metrics

Scheduling efficiency was assessed using five performance measures. Mean waiting time was the calculated length of time taken by processes in the read queue before they were executed. Average turnaround time was an indicator of the overall time that it takes to complete a process. Average response time was used to measure the time at which the initial CPU was allocated. The CPU usage was estimated as the percent of the time that the CPU was busy running tasks. Lastly, context switching was also being measured in order to quantify scheduling overhead, as a high context switching causes ineffective utilization of computer processing time.

4 Output and Result Analysis

Prior to running the detailed experimental analysis, the following implementation was run against the default configuration. This initial run generated a small number of processes with the inbuilt process generator. The generator develops the processes that have random arrival times, burst times as well as priority values in a predetermined range of values. The burst time prediction model was initially trained by using the generated processes. Their implementation makes use of a Decision Tree Regressor to learn process attribute to burst time behaviour relationships. The model uses the arrival time and priority of each process as the feature of the input and forecasts the CPU burst duration. The AI-Enhanced ADRR scheduler makes use of these predicted burst times to dynamically change scheduling behaviour. The scheduler tries to assign time quanta more optimally and minimise unnecessary context switches by adding in prediction. After prediction finishes, forecasting models are implemented consecutively on one set of processes. The Round Robin traditionally used scheduler is run first with fixed time quantum of four time units. The algorithm also selects each process in the ready queue and spends a maximum of the set quantum time with it before preempting it and putting it back at the end of the queue in case the process has not ended. In the process of execution, the scheduler logs response time, waiting time, turnaround time and CPU utilization and switches between contexts. After the execution of the Round Robin, the AI-Enhanced Adaptive Dynamic Round Robin scheduler is performed on the same workload. Contrary to the physical Round Robin scheduling, the algorithm is dynamically computed by dynamically determining the time quantum as per the anticipated burst times of the existing processes in the ready queue.

RR vs AI-ADRR Performance Comparison

Metric	Round Robin	AI-Enhanced ADRR
Avg Waiting Time	81.00	72.75
Avg Turnaround Time	94.17	85.92
Avg Response Time	18.42	34.75
CPU Utilisation (%)	100.00	100.00
Context Switches	31	18

Improvements (AI-ADRR over RR)

Waiting Time Reduction: 10:19%

Turnaround Time Reduction: 8.76%

Context Switch Reduction: 41.94%

Figure 1: Baseline execution output of the OS Scheduler comparing Round Robin and AI-Enhanced ADRR algorithms.

The baseline output shows the basic behaviour of the introduced scheduling structure. As the two algorithms both work on the workload, variations in performance measures directly translate into the effectiveness of the strategy of scheduling. In the normal executions, AI-Enhanced ADRR scheduler makes the context switch noticeable but does not affect the waiting and turnaround time across regular or slightly better. This is reduced due to the fact that the adaptive scheduler more intelligently assigns the CPU time depending on the expected behaviour of the process.

4.1 Effect of Workload Size

Workload size represents the number of processes present in the system. As the number of processes increases, the scheduling algorithm must manage longer ready queues and higher contention for CPU resources.

1) **Workload Size = 1000**

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	281951.51	281821.88	21698.52
Avg Turnaround Time	282246.51	28116.88	281993.52
Avg Response Time	1994.87	3989.54	3980.88
CPU Utilisation (%)	99.96	99.96	99.96
Context Switches	72750.00	60317.00	60720.00

2) **Workload Size = 50**

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	6457.24	6406.42	6464.18
Avg Turnaround Time	6684.88	6637.06	6691.82
Avg Response Time	68.88	173.04	146.70
CPU Utilisation (%)	99.78	99.78	99.78
Context Switches	2803.00	1133.00	1754.00

3) **Workload Size = 500**

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	133321.97	132945.39	133014.76
Avg Turnaround Time	133611.97	133235.39	133304.76
Avg Response Time	994.86	2487.69	2476.42
CPU Utilisation (%)	99.93	99.93	99.93
Context Switches	35750.00	21388.00	28145.00

Figure 2: Effect of Workload Size on Scheduling Metrics

The workload size is an amount of processes within the system. With more processes, the scheduling algorithm is faced with longer ready lines as well as increased competition by the CPU resources. According to the experimental findings, Round Robin algorithm introduced the maximum number of switches in con-text at the expense of the workload size. As an example, with high workloads, RR generates about 35,750 switches per context, whereas under heavy workloads, ADRR generates about 21,388 switches and AI-ADRR generates about 28,145 switches. According to the experimental findings, Round Robin algorithm introduced the maximum number of switches in con-text at the expense of the workload size. As an example, in case of heavy workloads, RR generates about 35,750 context switches. Compared to that, ADRR ensures this figure is only about 21,388 switches, whereas AI-ADRR plans about 28,145 switches. This difference is caused by differences in terms of scheduling. Round robin is defined as having a fixed time quantum. The preemptation of forcing processes is often done when it only needs slightly longer execution time. This creates an over overhead of context switching. Conversely, ADRR has time quantum dynamically adjusted to the current work load characteristics of precise processes that can take longer duration where needed, hence reducing preemption.The waiting time results further demonstrate the efficiency difference:

i) RR Waiting Time ≈ 133,321 ii) ADRR Waiting Time ≈ 132,945 iii) AI-ADRR Waiting Time ≈ 133,014

RR yields the shortest response time (around 994) as processes are given the CPU access quite fast as they are frequently rotated. Nonetheless, adaptive algorithms allow processes to be executed longer once started accumulate higher response times (about 2487) in ADRR and AI-ADRR. CPU occupancy is also high at all times (0.999 percent) because the CPU is busy at all times different scheduling algorithms.

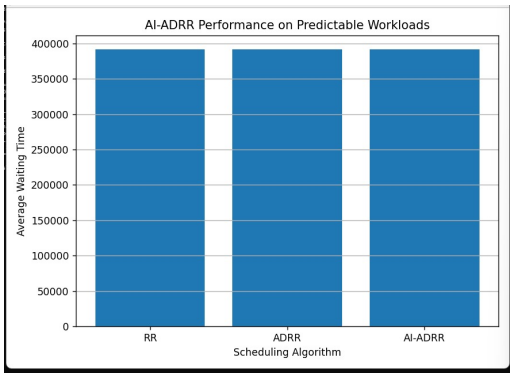

Figure 3: Graphical Comparison of Scheduling Performance for Different Workload Sizes

The graph also qualifies that ADRR has lower waiting and turnaround times than those of RR and AI-ADRR with an increase in workload. Adaptive time quantum allows ADRR to minimize the unexplained context switches without losing out on fairness amongst processes. The adaptive behavior ensures that the performance of sta- ble scheduling remains constant as the size of the workload increases, and excessive scheduling overhead so commonly seen with the Round Robin implementation does not occur.

4.2 Effect of Burst Time Variance

Burst time variance measures how much process execution times differ from one another. Systems with heterogeneous workloads contain processes with widely varying burst times, making scheduling decisions more complex.

Low Variance Dataset
Burst Mean: 300.0
Burst Std Dev: 0.0
Burst Min: 300
Burst Max: 300

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	12246.24	12129.64	12246.24
Avg Turnaround Time	12546.24	12429.64	12546.24
Avg Response Time	95.52	239.88	95.52
CPU Utilisation (%)	99.83	99.83	99.83
Context Switches	3700.00	1482.00	3700.00

Moderate Variance Dataset
Burst Mean: 217.14
Burst Std Dev: 125.06
Burst Min: 0
Burst Max: 385

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	5882.88	5785.40	5785.40
Avg Turnaround Time	6019.22	6002.54	6002.54
Avg Response Time	69.24	173.16	173.16
CPU Utilisation (%)	99.77	99.77	99.77
Context Switches	2686.00	1070.00	1070.00

High Variance Dataset
Burst Mean: 458.84
Burst Std Dev: 595.35
Burst Min: 1
Burst Max: 1500

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	6925.52	6908.32	6913.40
Avg Turnaround Time	7384.36	7367.16	7372.24
Avg Response Time	59.08	148.46	148.46
CPU Utilisation (%)	99.89	99.89	99.89
Context Switches	5693.00	2264.00	4266.00

Figure 4: Effect of Burst Time Variance on Scheduling Algorithms

Low Variance Scenario

When burst times are nearly identical, the impact of scheduling strategy becomes less significant. In this scenario:

i) RR Waiting Time ≈ 12246 ii) ADRR Waiting Time ≈ 12129 iii) AI-ADRR Waiting Time ≈ 12246

Because all processes require similar execution time, adaptive adjustments offer limited advantages. However, ADRR still significantly reduces context switches:

i) RR: 3700 ii) ADRR: 1482 ii) AI-ADRR: 3700

Moderate Variance Scenario

When burst time variance increases, adaptive algorithms begin to show stronger performance improvements.

i) RR Waiting Time ≈ 5802 ii) ADRR Waiting Time ≈ 5785 iii) AI-ADRR Waiting Time ≈ 5785

Context switching is significantly reduced for adaptive algorithms:

i) RR: 2686 ii) ADRR: 1070 iii) AI-ADRR: 1070

High Variance Scenario

High variance represents the most challenging scheduling scenario. Here, burst times vary widely among processes.

i) RR Waiting Time ≈ 6925 ii) ADRR Waiting Time ≈ 6908 iii) AI-ADRR Waiting Time ≈ 6913

Context switching differences become much more pronounced:

i) RR: 5693 ii) ADRR: 2264 iii) AI-ADRR: 4266

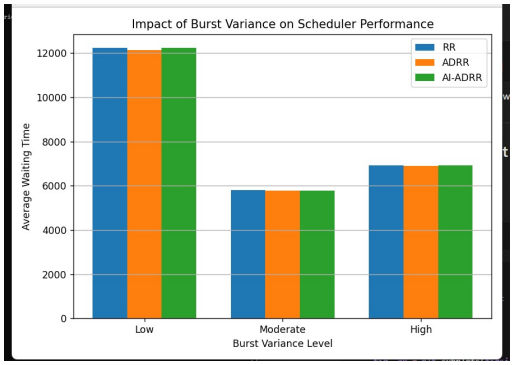

Figure 5: Graphical Representation of Burst Variance Impact

These findings show that adaptive algorithms are more effective in terms of managing heterogeneous workloads, through dynamic adjustment of their scheduling behavior. Adaptive algorithms reduce wasted preemptions and decrease the context switching overhead by adjusting the amount of time quantum depending on properties of processes in the ready queue. This gives the bet- ter scheduler the ability to service processes that have highly different bursts.

4.3 Effect of Time Quantum

The time quantum is one of the most critical parameters affecting the performance of Round Robin scheduling.

Experiments were conducted using time quantum values of: $\mathbf{2, 4, 8, 16, 32}$

1) RR Quantum = 2

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	133798.28	132945.39	133014.76
Avg Turnaround Time	134888.28	133235.39	133304.76
Avg Response Time	497.32	2487.69	2476.42
CPU Utilisation (%)	99.93	99.93	99.93
Context Switches	72000.00	21388.00	28145.00

2) RR Quantum = 4

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	133321.97	132945.39	133014.76
Avg Turnaround Time	133611.97	133235.39	133304.76
Avg Response Time	994.86	2487.69	2476.42
CPU Utilisation (%)	99.93	99.93	99.93
Context Switches	35750.00s	21388.00	28145.00

3) RR Quantum = 8

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	133366.00	132945.39	133014.76
Avg Turnaround Time	133656.00	133235.39	133304.76
Avg Response Time	1991.10	2487.69	2476.42
CPU Utilisation (%)	99.93	99.93	99.93
Context Switches	17875.00	21388.00	28145.00

4) RR Quantum = 16

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	131652.02	132945.39	133014.76
Avg Turnaround Time	131942.02	133235.39	133304.76
Avg Response Time	3984.66	2487.69	2476.42
CPU Utilisation (%)	99.93	99.93	99.93
Context Switches	8700.00	21388.00	28145.00

Figure 6: Effect of Time Quantum on Scheduling Performance

When the quantum is very small (Q = 2), RR produces extremely high context switching:

- Context Switches ≈ 72,000

This occurs because processes are preempted very frequently, leading to significant overhead.

As the quantum increases:

1) Q = 4 → 35,750 switches 2) Q = 8 → 17,875 switches 3) Q = 16 → 8,700 switches 4) Q = 32 → 4,350 switches

Although context switching decreases, response time increases slightly because processes must wait longer for CPU access. In contrast, ADRR and AI-ADRR maintain relatively stable performance across all quantum values:

i) ADRR Waiting Time ≈ 132,945 ii) AI-ADRR Waiting Time ≈ 133,014

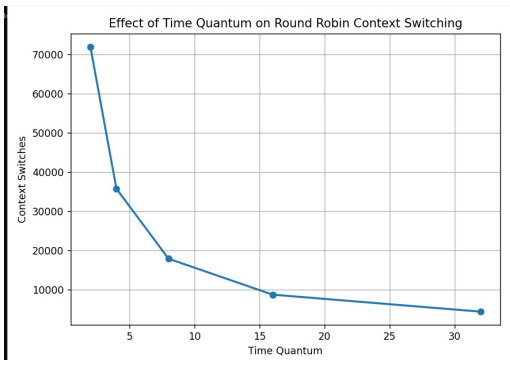

Figure 7: Graph of Performance Variation with Time Quantum

It means that adaptive algorithms are not so sensitive to the need to tune the parameters and are able to provide efficient scheduling without explicitly tuning the quantum. In contrast to the conventional Round Robin scheduling, when performance highly relies on the choice of an optimum time quantum, adaptive algorithms adaptively change the parameters of the scheduling according to the features of the workload. This flexibility allows the ability to maintain constant performance of the systems under various conditions.

4.4 Effect of Arrival Patterns

The arrival patterns affect the processes that arrive at the ready queue as time progresses. Three patterns were considered: 1) Uniform Arrival 2) Burst Arrival 3) Random Arrival

Uniform Arrival Dataset

Processes: 500

Arrival Min: 0

Arrival Max: 2495

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	136469.49	136148.76	136124.84
Avg Turnaround Time	136759.49	136438.76	136414.84
Avg Response Time	995.51	2488.07	2488.34
CPU Utilisation (%)	100.00	100.00	100.00
Context Switches	35750.00	17410.00	30086.00

Burst Arrival Dataset

Processes: 500

Arrival Min: 0

Arrival Max: 0

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	138962.00	138962.00	138962.00
Avg Turnaround Time	139252.00	139252.00	139252.00
Avg Response Time	998.00	998.00	998.00
CPU Utilisation (%)	100.00	100.00	100.00
Context Switches	35750.00	35750.00	35750.00

Random Arrival Dataset

Processes: 500

Arrival Min: 0

Arrival Max: 50

Metric	Round Robin	ADRR	AI-Enhanced ADRR
Avg Waiting Time	138961.92	138793.45	138796.15
Avg Turnaround Time	139251.92	139083.45	139086.15
Avg Response Time	995.60	2489.66	2489.66
CPU Utilisation (%)	100.00	100.00	100.00
Context Switches	35750.00	27976.00	27988.00

Figure 8: Effect of Process Arrival Patterns

Uniform Arrival

Uniform Processes arrive on a regular occasion. i) Waiting Time = 136k ii) Turnaround Time = 136k

Everything is similar as the load on the system does not vary.

Burst Arrival

All processes arrive together, with the resultant creation of maximum contention on the CPU resources.

i) Waiting Time = 138k ii) Turnaround Time = 139k

In this case, the discrepancies between algorithms are reduced since the scheduler is forced to deal with a huge ready queue at the moment.

Random Arrival

Processes are random and this is more realistic of the actual workloads.

i) RR Waiting Time 138.962 ii) ADRR Waiting Time 138.793 iii) AI-ADRR Waiting Time 138.796

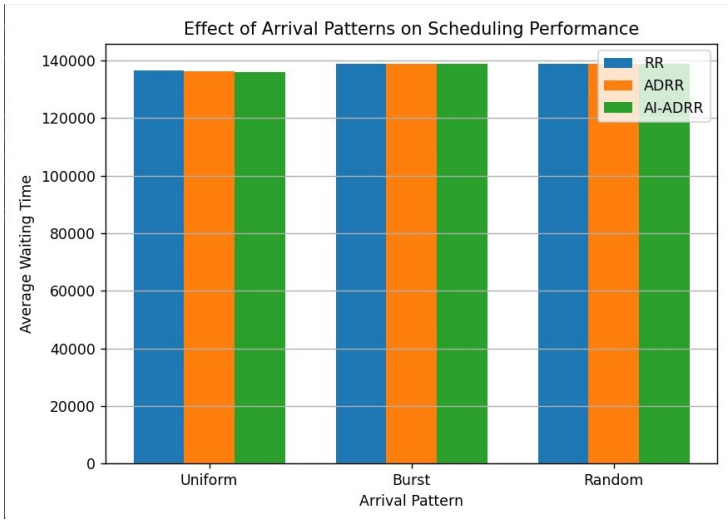

Figure 9: Graphical Comparison of Arrival Pattern Effects

The adaptive algorithms are a little more effective because they can change the decision to schedule dynamically as more processes enter the system. These algorithms accommodate the time quantum to the characteristics of the ready queue, and therefore give a more effective response to changes in the intensity of the workload and the behaviour of the process arrival.

4.5 Context Switching Comparison

Context switching overhead plays a major role in overall system efficiency.

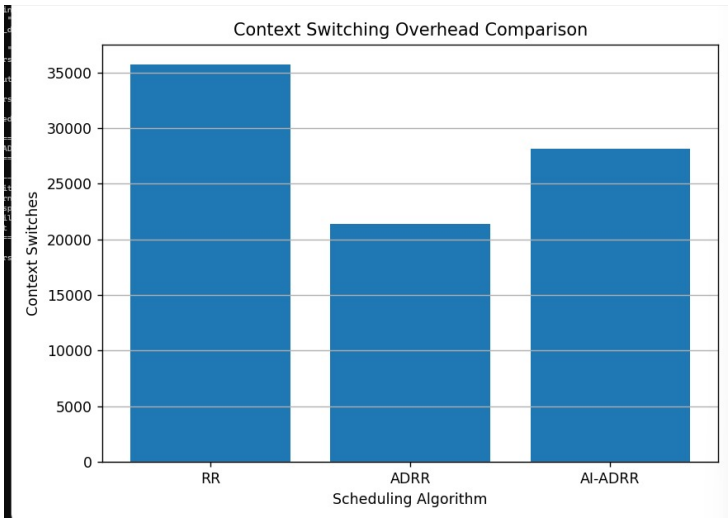

Figure 10: Comparison of Context Switches Across Scheduling Algorithms

The graph is clear to indicate that: RR generates most context switches, ADRR generates least switching overhead at all times, and AI-ADRR generates moderate switching behavior. Lessening context switching enhances CPU efficiency since less time is used in saving and restoring process states.

4.6 Overall Observations

Overall, in all experiments, some important patterns can be drawn:

- Round Robin is highly responsive but with too much context switching.
- ADRR also efficiently uses a lot of time because the quantum of time is dynamically adjusted.
- Balanced performance of AI-ADRR is due to the integration of the adaptive scheduling and predictive capabilities.
- CPU usage is always very high in all algorithms and this means that scheduling influences the latency and overhead, and not the usage of the processor.

These findings confirm that adaptive scheduling strategies offer enhanced overall performance in more modern systems, where workloads vary and are of heterogeneous nature.

5 Conclusion

This paper provided an in-depth analysis of three CPU scheduling algorithms Round Robin, Adaptive Dynamic Round Robin and AI-Enhanced ADRR. The algorithms were tested in conditions of multi- ple workloads with differences in the extent of the workload, variations in the burst time, time quantum dimensions, and variations in process entrance patterns. The findings indicate that traditional Round Robin scheduling is fair and has a high con-text switching overhead with high reliance on the time quantum chosen. In various experiments Round Robin had implemented over 35000 context switches, a fact that adds a lot of scheduling overhead to it. Adaptive Dynamic Round Robin showed better outcomes because it dynamically varied the behaviour of workload (scheduling). ADRR always minimized the overhead of context switch- ing and offered the same or slightly better waiting and turnaround time than Round Robin scheduling. AI-ADRR also improved performance of scheduling since predictive decision making was included in the system. The algorithm did not need wideCVF at times, and had a marginally higher context switching frequency than ADRR, however, it was also able to deal with dynamic workloads effectively and was able to work under all experimental conditions. Generally, this research shows that adaptive scheduling algorithms are more stable and efficient in current computing systems where workloads are dynamic and irregularly distributed. Future directions may be to consider practices of incorporating more advanced machine learning methods to be more efficient regarding CPU scheduling planning.